

Algorytmiczna Analiza Danych

Ćwiczenia 13

16 stycznia 2025 r.

Adrian Herda
Politechnika Wrocławska

1. Ćwiczenie 46 – Idea algorytmu Apriori

Algorytm Apriori - klasyczny algorytm służący do znajdowania częstych zbiorów elementów (frequent itemsets) w transakcyjnych bazach danych.

1.1. Właściwość Apriori (Anti-monotonicity)

Kluczowa dla działania algorytmu jest zasada:

Jeśli zbiór elementów jest częsty, to każdy jego podzbiór musi być częsty. I odwrotnie: jeśli zbiór jest rzadki, to każdy jego nadzbiór również jest rzadki.

Dzięki tej właściwości możemy drastycznie ograniczyć przestrzeń poszukiwań (tzw. *pruning*). Jeśli zbiór A, B jest rzadki, nie musimy sprawdzać A, B, C, ponieważ on również musi być rzadki.

1.2. Proces generowania kandydatów

Algorytm działa iteracyjnie (poziomami $k = 1, 2, \dots$):

1. **Łączenie (Join)** - Kandydaci o rozmiarze k (C_k) są generowani poprzez łączenie częstych zbiorów o rozmiarze $k - 1$ (L_{k-1}). Łączymy dwa zbiory $X, Y \in L_{k-1}$ tylko wtedy, gdy ich pierwsze $k - 2$ elementy są identyczne (zakładając porządek leksykograficzny).
2. **Przycinanie (Prune)** - Dla każdego wygenerowanego kandydata sprawdzamy, czy **wszystkie** jego podzbiory o rozmiarze $k - 1$ znajdują się w L_{k-1} . Jeśli którykolwiek podzbiór nie jest częsty, kandydat jest usuwany zgodnie z właściwością Apriori.

2. Ćwiczenie 47 – Oznaczenie węzłów kraty (M, D, C, R)

Dane:

- Liczba transakcji (N): 10
- Próg wsparcia: 30% $\rightarrow$ 3 transakcje.

Analiza licznosci zbiorów (Support count):

Zbiór	Liczność	Status
{a}	5	D
{b}	7	D
{c}	5	D

Zbiór	Liczność	Status
{d}	9	D
{e}	6	C
{ab}	3	M
{ac}	2	R
{ad}	4	C
{ae}	4	C
{bc}	3	M
{bd}	6	D
{be}	4	C
{cd}	4	M
{ce}	2	R
{de}	6	D
{abc}	1	R
{abd}	2	R
{abe}	2	R
{acd}	1	R
{ace}	1	R
{ade}	4	M
{bcd}	2	R
{bce}	1	R
{bde}	4	M
{cde}	2	R
{abcd}	0	R
{abce}	0	R
{abde}	2	R
{acde}	1	R
{bcde}	1	R
{abcde}	0	R

Oznaczenia:

- **M (Maksymalny)** - Częsty, brak częstych nadzbiorów.
- **D (Domknięty)** - Częsty, brak nadzbiorów o *tym samym* wsparciu (Maksymalne też są domknięte, ale tutaj rozróżniamy je dla precyzji, przypisując M do tych na granicy „częstości”).
- **C** - Częsty, nie M i nie D.
- **R (Rzadki)** - Poniżej progu.

Wynikowa klasyfikacja węzłów:

- **M:** ab, bc, cd, ade, bde
- **D:** a, b, c, d, bd, de
- **C:** e, ad, ae, be
- **R:** ac, ce oraz wszystkie zbiory 3-elementowe inne niż ade, bde i wszystkie 4-elementowe.

3. Ćwiczenie 48 – Intuicje: Maksymalne vs Domknięte

3.1. Maksymalny zbiór częsty:

- **Intuicja:** Jest to „granica” zbiorów częstych. Najdłuższe możliwe kombinacje, które są jeszcze częste.
- **Cel:** Kompaktowa reprezentacja wszystkich częstych zbiorów. Jeśli znamy zbiory maksymalne, wiemy, że każdy ich podzbiór też jest częsty.
- **Wada:** Tracimy informację o dokładnej liczności podzbiorów (kompresja stratna).

3.2. Domknięty zbiór częsty:

- **Intuicja:** Zbiór, który „maksymalizuje” przedmioty dla danej grupy transakcji. Dodanie czegokolwiek do tego zbioru spowodowałoby spadek wsparcia (występowanie w mniejszej liczbie transakcji).
- **Cel:** Pozwala na odtworzenie dokładnego wsparcia wszystkich częstych podzbiorów bez skanowania bazy (kompresja bezstratna). Wsparcie dowolnego zbioru X jest równe wsparciu jego najmniejszego domkniętego nadzbioru.

Relacja:

$$\text{Maksymalne} \subseteq \text{Domknięte} \subseteq \text{Częste} \quad (3.1)$$

Każdy maksymalny zbiór jest domknięty, ale nie każdy domknięty jest maksymalny.

4. Ćwiczenie 49 – Złożoność Algorytmu Apriori

Niech d to liczba unikalnych przedmiotów, N to liczba transakcji, a w to maksymalna szerokość transakcji.

1. **Złożoność czasowa:** Głównym kosztem jest zliczanie wsparcia dla kandydatów C_k . W najgorszym przypadku algorytm generuje $O(2^d)$ kandydatów. Koszt zliczania to:

$$O\left(\sum_k |C_k| \cdot N \cdot w\right) \quad (4.2)$$

W praktyce często upraszcza się to do $O(2^d)$, co wskazuje na wykładniczą złożoność względem liczby produktów.

2. **Złożoność pamięciowa:** Wymaga przechowywania kandydatów na danym poziomie oraz liczników.

$$O\left(\max_k |C_k|\right) \quad (4.3)$$

W najgorszym przypadku (gdy $k \approx \frac{d}{2}$) pamięć rośnie wykładniczo.

Wpływ progu wsparcia ($\text{Support}_{\min}$): Tak, próg ma krytyczny wpływ.

- **Zmniejszanie $\text{Support}_{\min}$:** Powoduje gwałtowny wzrost liczby częstych zbiorów i kandydatów.
- **Negatywne konsekwencje:** Wykładniczy wzrost czasu obliczeń i zapotrzebowania na pamięć. Może dojść do wygenerowania tak dużej liczby kandydatów na poziomie 2 (pary), że algorytm nie zmieści się w pamięci RAM.